Alternative Assessment Tool (AAT) Report on

"Design and Verification of Combinational and Sequential Digital Modules using Verilog HDL"

Submitted in partial fulfilment of the requirements of the Alternative Assessment Tool (AAT) for the course

ADVANCED DIGITAL AND LOGIC DESIGN (ADLD)
[Course Code: 23ECSPE1AD]

Submitted by

Paladugula Ajay Kumar
1BF24EC099

Submitted to

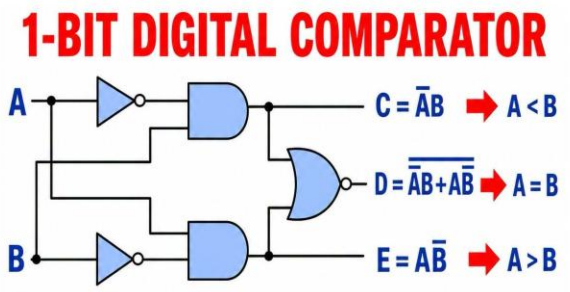
Prof. Ajaykumar Devarapalli

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
BMS College of Engineering
Bull Temple Road, Basavanagudi, Bengaluru-560019

Academic Year 2026
– 2027

SL NO.	CONTENTS
1.	COMPARATOR
2.	4-BIT PARITY CHECKER AND GENERATOR
3.	4-BIT BARREL SHIFTER
4.	TIMER MODULE
5.	RING COUNTER
6.	JOHNSON COUNTER
7.	SYNCHRONOUS COUNTER
8.	ASYNCHROMOUS COUNTER
9.	UNIVERSAL SHIFT REGISTER
10.	DIVIDE BY ‘N’ COUNTER
11.	MEALY TO DETECT 101
12.	MOORE TO DETECT 101
13.	RANDOM SEQUENCE GENERATOR(LFSR)

1.COMPARATOR:CIRCUIT DESIGN



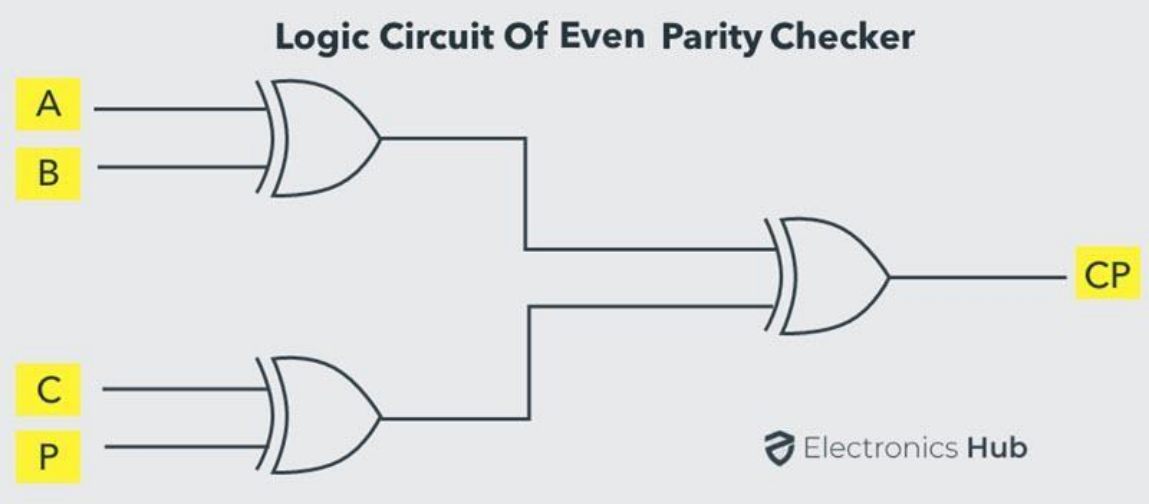
INPUTS		OUTPUTS		
B	A	A > B (E)	A = B (D)	A < B (C)
0	0	0	1	0
0	1	1	0	0
1	0	0	0	1
1	1	0	1	0

CODE:MAIN MODULE AND TESTBENCH

```
module comp1bit(input A, B, output Y1, Y2,
Y3);     assign Y1 = ~A & B;     assign Y2 = ~(A
^ B);     assign Y3 = A & ~B; endmodule

module tb;     reg A, B;     wire Y1,
Y2, Y3;     comp1bit uut(A, B, Y1,
Y2, Y3);
   initial begin
      $monitor("%0t: %b %b -> Y1=%b Y2=%b Y3=%b", $time, A, B, Y1, Y2, Y3);
      {A, B} = 2'b00; #10;
      {A, B} = 2'b01; #10;
      {A, B} = 2'b10; #10;
      {A, B} = 2'b11; #10;
      $finish;
end
endmodule
```

2.4-BIT PARITY GENERATOR AND CHECKER:CIRCUIT DESIGN



CODE:MAIN MODULE AND TESTBENCH

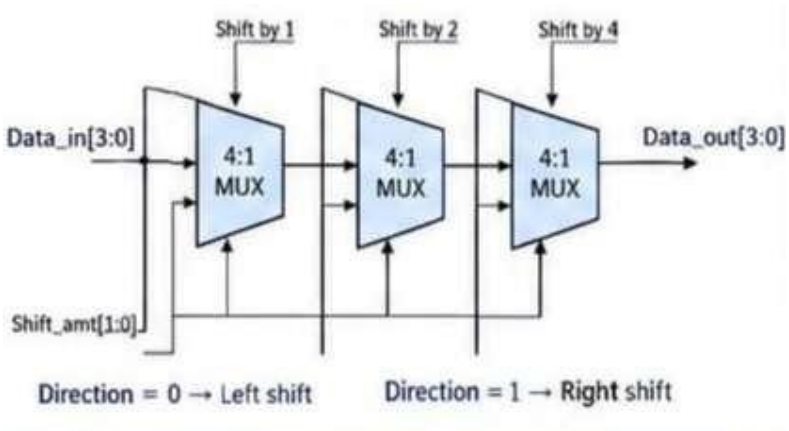
```
module parity_4bit(
input [3:0] Data,
input Parity_in,
output Parity_even,
output Parity_odd,
   output Parity_error
);
   assign Parity_even = ^Data;      assign
Parity_odd = ~Parity_even;      assign
Parity_error = Parity_even ^ Parity_in;
endmodule

module tb_parity;     reg [3:0] Data;     reg
Parity_in;     wire Parity_even, Parity_odd,
Parity_error;

   parity_4bit uut(Data, Parity_in, Parity_even, Parity_odd, Parity_error);

   initial begin
      $monitor("%0t: Data=%b P_in=%b -> P_even=%b P_odd=%b P_error=%b", $time,
Data, Parity_in, Parity_even, Parity_odd, Parity_error);
      Data = 4'b0000; Parity_in = 0; #10;
      Data = 4'b1010; Parity_in = 0; #10;
      Data = 4'b1010; Parity_in = 1; #10;
      Data = 4'b1110; Parity_in = 1; #10;
      Data = 4'b1110; Parity_in = 0; #10;
      $finish;
end
endmodule
```

3.4-BIT BARREL SHIFTER:CIRCUIT DESIGN



CODE:MAIN MODULE AND TESTBENCH

```
module
barrel_shifter_4bit(
input [3:0] Data_in,
input [1:0] Shift_amt,
input Direction,     output
reg [3:0] Data_out
);
   always @(*) begin
      if (Direction == 1'b0) // 0 = Left shift
Data_out = Data_in << Shift_amt;         else
// 1 = Right shift
         Data_out = Data_in >> Shift_amt;
end
endmodule

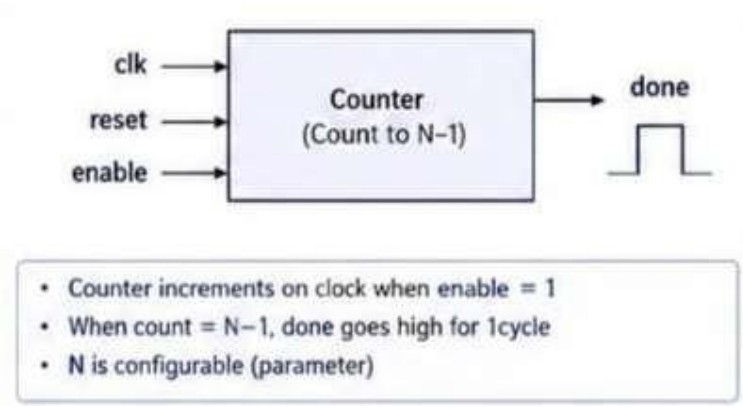
module
tb_barrel_shifter;     reg
[3:0] Data_in;     reg
[1:0] Shift_amt;     reg
Direction;
   wire [3:0] Data_out;

   barrel_shifter_4bit uut(Data_in, Shift_amt, Direction, Data_out);

   initial begin
      $monitor("%0t: In=%b Dir=%b Amt=%d -> Out=%b", $time, Data_in, Direction,
Shift_amt, Data_out);
```

```
        Data_in = 4'b1101;
        Direction = 0; Shift_amt = 2'd0; #10;
        Direction = 0; Shift_amt = 2'd1; #10; // Left by 1
        Direction = 0; Shift_amt = 2'd2; #10; // Left by 2
        Direction = 1; Shift_amt = 2'd1; #10; // Right by 1
        Direction = 1; Shift_amt = 2'd2; #10; // Right by 2
        $finish;
end
endmodule
```

4.TIMER MODULE:CIRCUIT DESIGN



CODE:MAIN MODULE AND TESTBENCH

```
module ring_counter(
   input clk, rst,
   output reg [3:0] q
);
   always @(posedge clk or posedge rst) begin
if (rst)              q <= 4'b0001; // Initialize with a
single '1'        else
          q <= {q[2:0], q[3]}; // Shift left and circulate MSB to LSB
end
endmodule

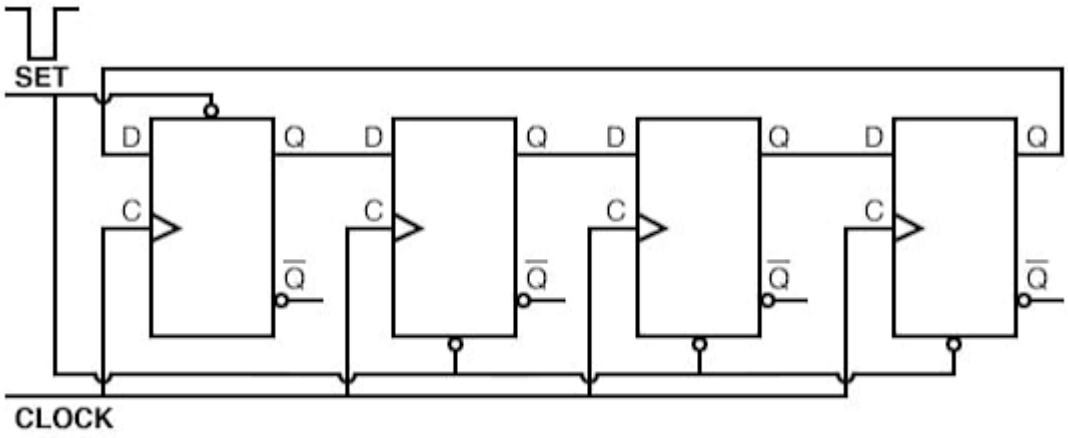
module tb_ring_counter;
   reg clk = 0, rst;
   wire [3:0] q;

   ring_counter uut(clk, rst, q);

   always #5 clk = ~clk; // 10ns clock period
```

```
    initial begin
        $monitor("%0t: rst=%b -> q=%b", $time, rst, q);
rst = 1; #10;
        rst = 0; #60; // Let it circulate
        $finish;
end
endmodule
```

5.RING COUNTER:CIRCUIT DESIGN



Set one stage, clear three stages

CODE:MAIN MODULE AND TESTBENCH

```
module ring_counter_schematic(
    input CLK, ORI,
    output reg [3:0] Q
);
    // Q[0] is FF-0, Q[1] is FF-1, Q[2] is FF-2, Q[3] is FF-3
    always @(posedge CLK or posedge ORI) begin
if (ORI)
            Q <= 4'b0001; // Preset FF-0, Clear FF-1, FF-2, FF-3
else
            Q <= {Q[2:0], Q[3]}; // D3=Q2, D2=Q1, D1=Q0, D0=Q3
end
endmodule

module tb_ring_schematic;
reg CLK = 0, ORI;
```

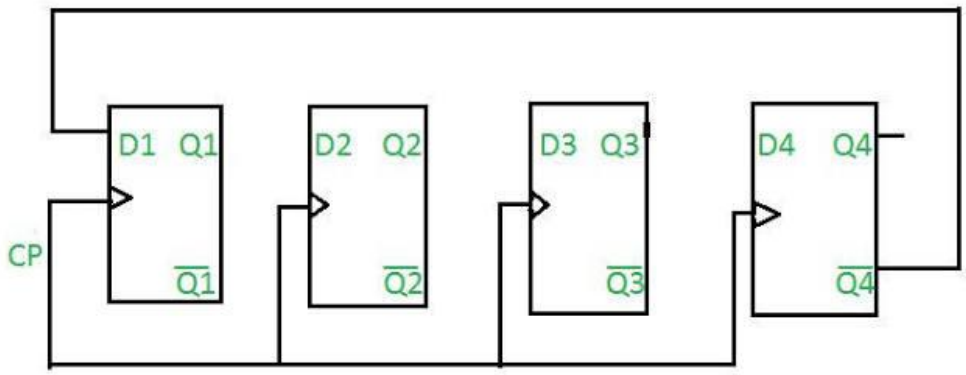
```
    wire [3:0] Q;

    ring_counter_schematic uut(CLK, ORI, Q);

    always #5 CLK = ~CLK;

    initial begin
        $monitor("%0t: CLK=%b ORI=%b -> Q(3:0)=%b", $time, CLK, ORI, Q);
        ORI = 1; #10;
        ORI = 0;
#50;          $finish;
end
endmodule
```

6.JOHNSON COUNTER:CIRCUIT DESIGN



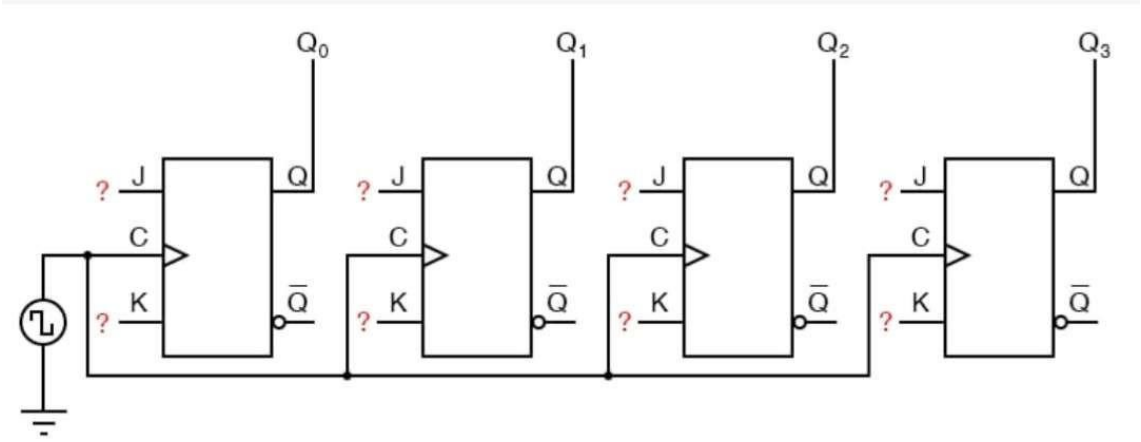
CODE:MAIN MODULE AND TESTBENCH

```
module johnson_counter(
    input clk, reset,
    output reg [3:0] q // Represents {D, C, B, A}
);
    always @(posedge clk or posedge reset)
begin         if (reset)              q <= 4'b0000;
else
            q <= {q[2:0], ~q[3]}; // Shift left, inverted MSB (D) feeds to LSB (A)
end
endmodule

module tb_johnson_counter;
    reg clk = 0, reset;     wire [3:0] q;
johnson_counter uut(clk, reset, q);
```

```
always #5 clk = ~clk;     initial
begin
        $monitor("%0t: clk=%b reset=%b -> q(DCBA)=%b", $time, clk, reset,
q);         reset = 1; #10;          reset = 0; #80; // Allow 8 clock cycles to
complete the sequence
        $finish;
end
endmodule
```

7.SYNCHRONOUS COUNTER:CIRCUIT DESIGN



CODE:MAIN MODULE AND TESTBENCH

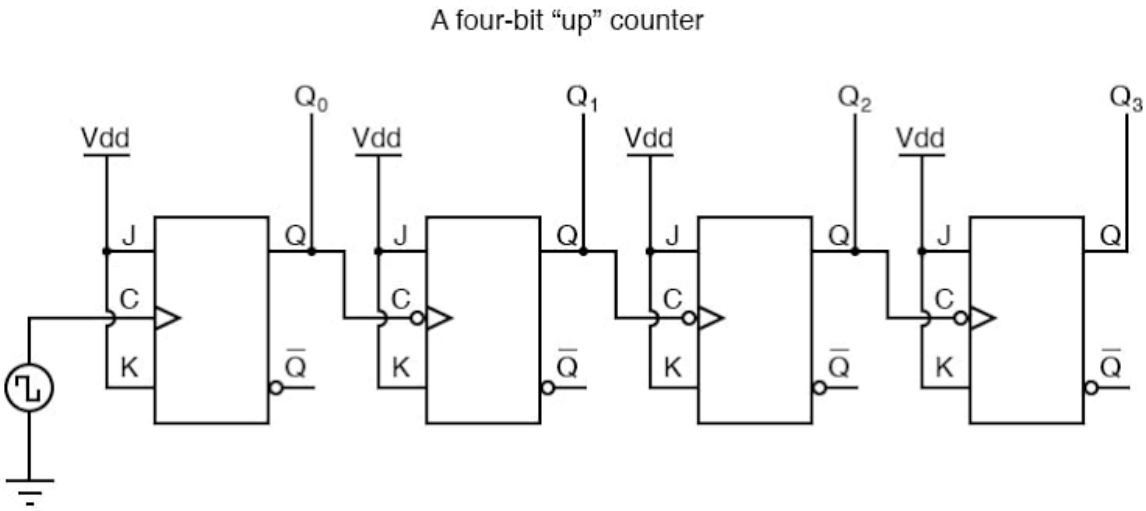
```
module sync_counter(
input clk, reset,
   output reg [3:0] q
);
   always @(posedge clk or posedge reset) begin
if (reset)

 q <= 4'b0000;
       else
  q <= q + 1; // All bits update synchronously on the clock edge
end
endmodule

module tb_sync_counter;
   reg clk = 0, reset;     wire
[3:0] q;     sync_counter uut(clk,
reset, q);     always #5 clk =
~clk;
   initial begin
       $monitor("%0t: clk=%b reset=%b -> q=%b", $time, clk, reset,
q);         reset = 1; #10;          reset = 0; #160; // Allow enough time to
count 0 to 15
```

```
        $finish;
end
endmodule
```

8.ASYNCHRONOUS COUNTER:CIRCUIT DESIGN



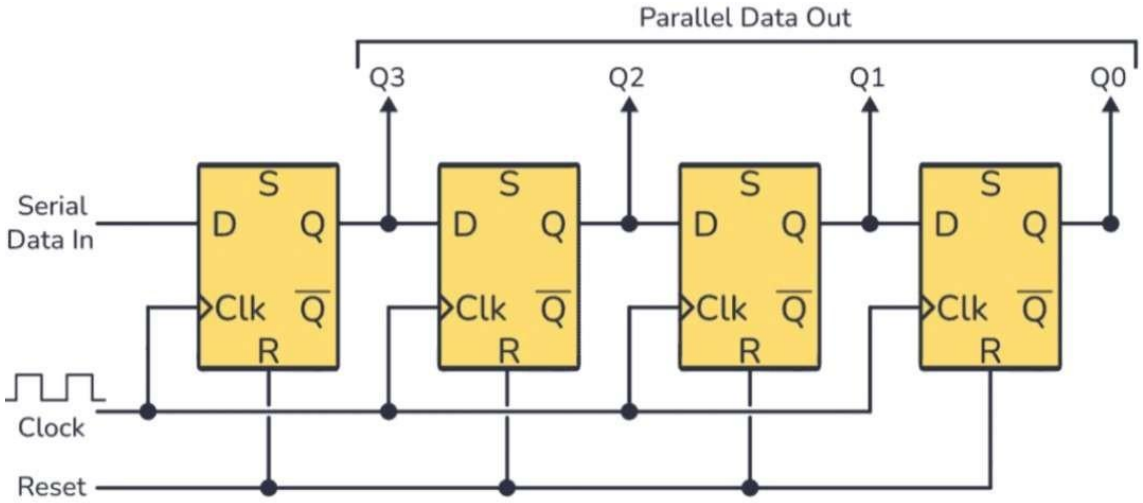
CODE:MAIN MODULE AND TESTBENCH

```
module async_counter_short(input clk, rst, output reg [3:0] q);
   // Each bit toggles on the negative edge of the previous bit (ripple clock)
always @(negedge clk  or posedge rst) if (rst) q[0] <= 0; else q[0] <= ~q[0];
always @(negedge q[0] or posedge rst) if (rst) q[1] <= 0; else q[1] <= ~q[1];
always @(negedge q[1] or posedge rst) if (rst) q[2] <= 0; else q[2] <= ~q[2];
always @(negedge q[2] or posedge rst) if (rst) q[3] <= 0; else q[3] <= ~q[3];
endmodule

module tb_async_short;     reg clk =
0, rst;     wire [3:0] q;
async_counter_short uut(clk, rst, q);
always #5 clk = ~clk;
   initial begin
      $monitor("%0t: clk=%b rst=%b -> q=%b", $time, clk, rst,
q);        rst = 1; #10;          rst = 0; #160;
      $finish;

 end
endmodule
```

9.UNIVERSAL SHIFT REGISTER:CIRCUIT DESIGN(SIPO)



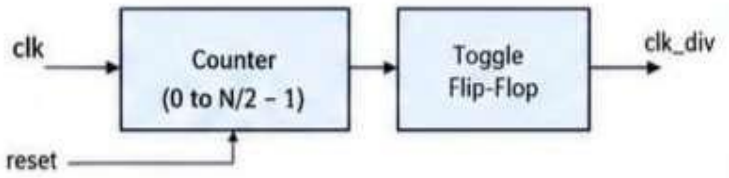
CODE:MAIN MODULE AND TESTBENCH

```
module sipo_shift_reg(     input clk, reset, serial_in,
output reg [3:0] q // Represents {Q3, Q2, Q1, Q0}
);
   always @(posedge clk or posedge reset) begin         if (reset)              q
<= 4'b0000;         else              q <= {serial_in, q[3:1]}; // Shift right:
serial_in to Q3, Q3 to Q2, etc.
   end
endmodule

module tb_sipo;
   reg clk = 0, reset, serial_in;     wire [3:0]
q;     sipo_shift_reg uut(clk, reset,
serial_in, q);     always #5 clk = ~clk;
initial begin
       $monitor("%0t: reset=%b sin=%b -> q(Q3-Q0)=%b", $time, reset, serial_in,
q);         reset = 1; serial_in = 0; #10;              reset = 0;
       serial_in = 1; #10; // Shift in 1
serial_in = 0; #10; // Shift in 0
serial_in = 1; #10; // Shift in 1
       serial_in = 1; #20; // Shift in 1, then 1

       $finish;
end
endmodule
```

10.DIVIDE BY ‘N’ COUNTER:CIRCUIT DESIGN



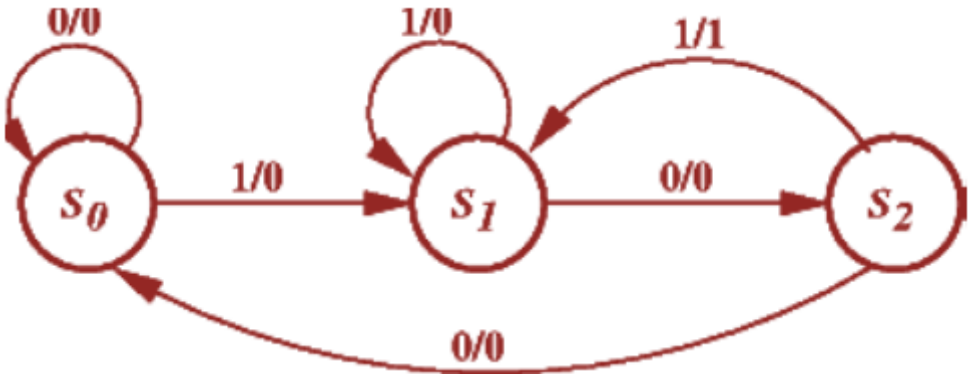
- Counter counts N/2 clock cycles
- Toggles output to get N division (50% duty cycle)
- N is parameterizable (even number)

CODE:MAIN MODULE AND TESTBENCH

```
module divide_by_n #(parameter N = 4)(
   input clk, reset,
   output reg clk_div
);
   reg [31:0] count;     always @(posedge clk or
posedge reset) begin         if (reset) begin
count <= 0;             clk_div <= 0;         end else
if (count == (N/2) - 1) begin
          count <= 0;
clk_div <= ~clk_div;
end else begin             count
<= count + 1;
       end
   end
endmodule

module tb;
   reg clk = 0, reset;     wire clk_div;
divide_by_n #(.N(4)) uut(clk, reset, clk_div);
always #5 clk = ~clk;
   initial begin
      $monitor("%0t: reset=%b -> clk_div=%b", $time, reset,
clk_div);         reset = 1; #10;          reset = 0; #100;          $finish;
end
endmodule
```

11.MEALY FSM:STATE DIAGRAM(101)



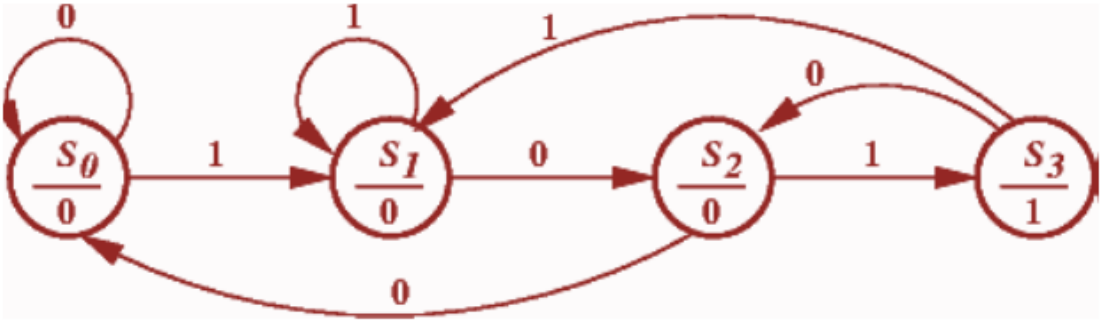
CODE:MAIN MODULE AND TESTBENCH

```
module mealy_101(
   input clk, rst, in,
   output reg out
);

 reg [1:0] state, next;     always
@(posedge clk or posedge rst)         if
(rst) state <= 2'b00;
       else state <=
next;     always @(*)
begin         next =
2'b00;          out = 0;
case (state)
          2'b00: next = in ? 2'b01 : 2'b00;        // State a
          2'b01: next = in ? 2'b01 : 2'b10;        // State b
          2'b10: begin next = 2'b00; out = in; end // State c (Detects 101)
endcase
   end
endmodule

module tb_mealy_101;
   reg clk = 0, rst, in;     wire out;
mealy_101 uut(clk, rst, in, out);
always #5 clk = ~clk;
   initial begin
      $monitor("%0t: in=%b state=%b -> out=%b", $time, in, uut.state, out);
rst = 1; in = 0; #10;
       rst = 0;
       in = 1; #10; // Detect 1 (State a -> b)
in = 0; #10; // Detect 0 (State b -> c)         in = 1;
#10; // Detect 1 (State c -> a, out = 1)
       in = 0; #10;
$finish;     end
endmodule
```

12.MOORE FSM:STATE DIAGRAM(101)



CODE:MAIN MODULE AND TESTBENCH

```
module moore_101(
   input clk, rst, in,
   output out
);
   reg [1:0] state, next;     parameter a = 2'b00, b =
2'b01, c = 2'b10, d = 2'b11;     always @(posedge clk
or posedge rst)
       if (rst) state <= a;
else state <= next;
   always @(*) begin         case (state)            a: next
= in ? b : a;            b: next = in ? b : c;            c: next
= in ? d : a;            d: next = in ? b : c;            default:
next = a;         endcase     end     assign out = (state ==
d); // Output is 1 only in state d endmodule

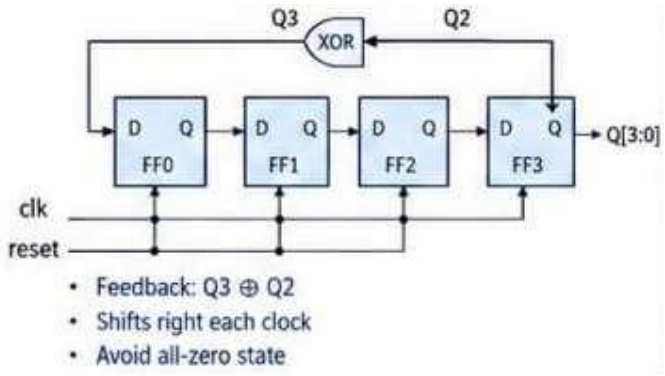
module tb_moore_101;
   reg clk = 0, rst, in;     wire out;
moore_101 uut(clk, rst, in, out);
always #5 clk = ~clk;     initial
begin
       $monitor("%0t: in=%b state=%b -> out=%b", $time, in, uut.state, out);
rst = 1; in = 0; #10;
       rst = 0;



in = 1; #10; // State a -> b in = 0;
#10; // State b -> c in = 1; #10; //
State c -> d (out = 1) in = 0; #10;
// State d -> c
```

```
        $finish;
end
endmodule
```

13.RANDOM SEQUENCE GENERATOR:CIRCUIT DESIGN(LFSR)

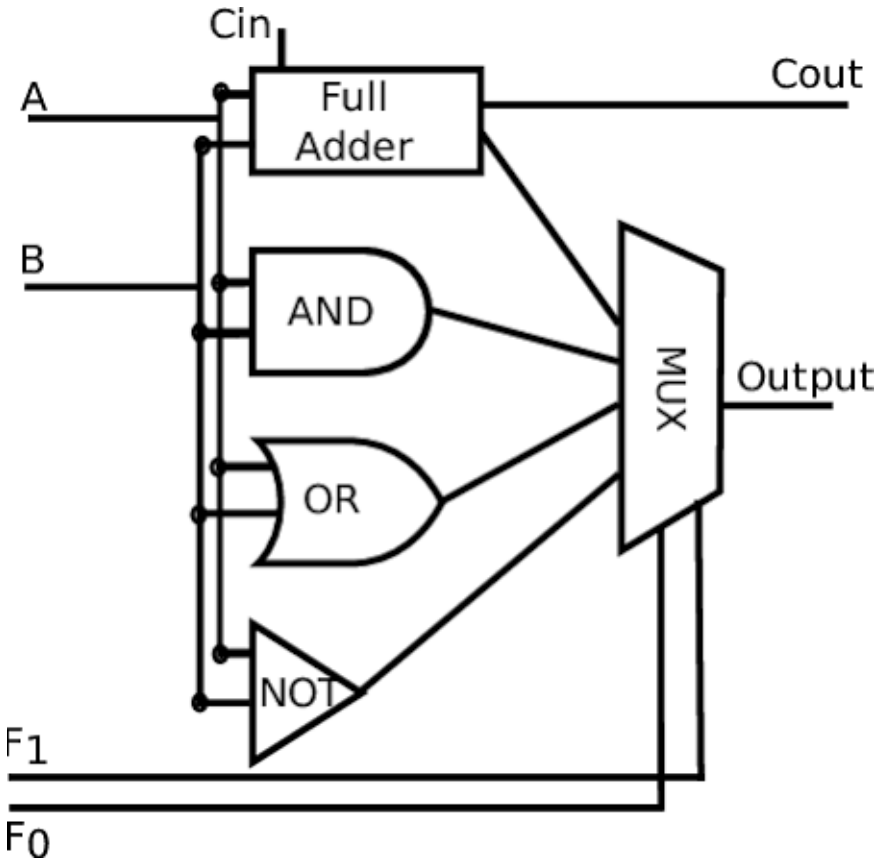


CODE:MAIN MODULE AND TESTBENCH

```
module lfsr_4bit(
input clk, reset,
   output reg [3:0] Q
);
   always @(posedge clk or posedge reset) begin
if (reset)
        Q <= 4'b0001; // Must initialize to a non-zero value to avoid lockup[cite: 14]
else
        // Q[0] gets Q[3] ^ Q[2], and others shift right[cite: 14]
        Q <= {Q[2], Q[1], Q[0], Q[3] ^ Q[2]};
end
endmodule

module tb;
   reg clk = 0, reset;     wire
[3:0] Q;    lfsr_4bit uut(clk,
reset, Q);     always #5 clk
= ~clk;     initial begin
       $monitor("%0t: reset=%b -> Q(FF3-FF0)=%b", $time, reset, Q);         reset =
1; #10;         reset = 0; #150; // Allow enough time to generate the pseudo-random
sequence
       $finish;
end
endmodule
```

14.ARITHEMATIC LOGIC UNIT:CIRCUIT DESIGN(1-BIT)



CODE:MAIN MODULE AND TESTBENCH

```
module alu_1bit_ (     input
A0, B0, Cin, M1, M0,
output F0, Cout
);
  // Compute Carry Out
   assign Cout = (A0 & B0) | (B0 & Cin) | (Cin & A0);
   // Combine operations into a 4-bit array and use {M1, M0} to select the output
   // Index 3: XOR, Index 2: OR, Index 1: AND, Index 0: SUM
wire [3:0] ops = {A0 ^ B0, A0 | B0, A0 & B0, A0 ^ B0 ^ Cin};
   // Output selected by multiplexer
   assign F0 = ops[{M1, M0}];

endmodule

module tb;     reg A, B, C, M1, M0;
wire F, Cout;     alu_1bit uut (A, B, C,
M1, M0, F, Cout);     initial begin
       $monitor("M1M0=%b%b | A=%b B=%b Cin=%b | F=%b Cout=%b", M1, M0, A, B, C,
F, Cout);
       {A, B, C, M1, M0} = 5'b110_00; #10; // Addition (Mux 00)
       {A, B, C, M1, M0} = 5'b110_01; #10; // AND (Mux 01)
```

```
        {A, B, C, M1, M0} = 5'b110_10; #10; // OR (Mux 10)
{A, B, C, M1, M0} = 5'b110_11; #10; // XOR (Mux 11)
end
endmodule
```